

Git Lifecycle & Pull Request Assignment

Comprehensive end-to-end repository challenge: local branching, patch staging, conflict triage, and upstream PR delivery.

Target Central Repository: https://github.com/Sukhilkumarnatha/DevOps_Tasks.git

Submission Rule: All files must reside strictly inside an isolated folder: `submissions/<github_username>/`. Never modify root files or peers' directories. Submit work as a Pull Request comparing your feature branch across forks to `upstream:main`.

PHASE 1: FORKING, UPSTREAM DUAL-WIRING & FEATURE BRANCHING

TASKS 01-02

Task 01: Remote Architecture Dual-Wiring

REMOTE SETUP

Fork the central repository to your personal GitHub account. Clone your fork locally using Git Bash. Then, add a second remote reference named `upstream` pointing to Sukhil's primary repository URL, verifying both fetch and push paths for both remotes.

Hint: Investigate how `git clone` sets up `origin`, and use `git remote add` to attach the secondary endpoint. Audit active remotes with verbose listing flags.

Pass Criteria: Verbose remote inspection outputs 4 endpoints: two tracking `origin` (your fork) and two tracking `upstream`.

Task 02: Isolated Feature Branch Checkout

BRANCH CREATION

Rather than working on your local `main` branch, spawn and check out a dedicated feature branch formatted as `feature/git-practice-<your_username>` using modern Git context-switching syntax.

Hint: Look into modern branch switching commands (`git switch`) with creation flags rather than legacy checkout syntax.

Pass Criteria: Active branch pointer confirms you have switched to your personalized feature branch.

PHASE 2: DIRECTORY SCAFFOLDING & ATOMIC SCRIPT COMMIT

TASKS 03-04

Task 03: Workspace Scaffolding & Shell Script Provisioning

FILE OPS & PERMS

Create the directory `submissions/<your_username>/`. Inside it, create an executable health script `app_check.sh` that tests an HTTP status code against an API endpoint. Ensure file mode `100755` (executable) is set before tracking.

Hint: Use directory creation commands with parent flags, populate the shell script with a bash shebang, and apply execution permissions via symbolic notation (`+x`).

Pass Criteria: Short status confirms file presence; file metadata reflects executable mode.

Task 04: Conventional Atomic Commit

COMMIT STANDARDS

Stage `app_check.sh` and commit it using Conventional Commits notation with a `feat(...)` prefix explaining the addition of the health inspection utility.

Hint: Avoid generic commit phrases like "added files". Use standard type scopes like `feat(health):` with an imperative description.

Pass Criteria: Commit history records a single atomic commit with author details and message conforming to standard.

PHASE 3: PATCH-LEVEL STAGING & DUAL-TIER DIFF AUDITS

TASKS 05-06

Task 05: Selective Staging via Interactive Hunks

INDEX MANIPULATION

Open `app_check.sh`. Add two independent logic modifications separated by several blank lines: a banner echo statement at the top, and an explicit exit code at the bottom. Use interactive staging to stage *strictly the top banner*, leaving the bottom exit code unstaged.

Hint: Research the patch-level flag (`-p`) for the staging utility. Use hunk navigation keys to accept only the first modification block.

Pass Criteria: Working directory diff shows uncommitted changes, while staged index diff shows only the top banner block.

Task 06: Independent Staging Commits & Diff Comparison

DIFF FORENSICS

Commit your staged banner modification with a `feat:` label. Verify that the remaining exit code change still resides safely in your working directory. Then, stage and commit the exit code modification with a `fix:` prefix.

Hint: Observe how `git diff` inspects the working tree, whereas `git diff --staged` inspects the index.

Pass Criteria: Commit graph shows two separate, clean commits derived from a single file modification cycle.

Task 07: Labeled Stashing & Restoration Cycle

SHELVING

Simulate an urgent mid-task context switch. Create a draft report file `GIT_TASK_REPORT.md` with uncommitted notes. Shelf your incomplete edits into the stash stack with an explicit descriptive message, verify the working tree is pristine, inspect the stash list, and pop your shelved work back.

Hint: Explore `git stash push -m`, inspect using `git stash list`, and retrieve modifications using `git stash pop`.

Pass Criteria: Working directory transitions from dirty to clean and back; stash stack clears completely post-pop.

PHASE 5: DIVERGENT BRANCHES & MERGE CONFLICT TRIAGE

TASKS 08-09

Task 08: Intentional Branch Divergence Simulation

CONFLICT INDUCTION

Spawn a side branch named `conflict-sim`. In `app_check.sh`, alter the script header line to read "Version 2.0 - Team Alpha" and commit. Switch back to your feature branch, alter that exact same line to read "Version 2.0 - Team Beta", and commit. Trigger a merge conflict by merging `conflict-sim` into your feature branch.

Hint: Git cannot resolve conflicting edits made to identical file offsets across diverged branches.

Pass Criteria: Terminal outputs an explicit automatic merge failure warning, marking unmerged files in status.

Task 09: Conflict Marker Resolution & Branch Pruning

RESOLUTION

Open the conflicted script. Inspect conflict boundaries (`<<<<<< HEAD, =====, >>>>>>`). Manually unify the logic, delete all conflict markers, stage the resolved file, commit the merge, and delete the temporary `conflict-sim` branch.

Hint: Resolve by keeping the intended combined string, stage using standard add commands, and delete the merged branch with safe removal flags (`-d`).

Pass Criteria: Repository status returns to clean; commit graph displays a dedicated merge commit.

PHASE 6: CODE ARCHAEOLOGY & COMMIT FORENSICS

TASKS 10-11

Task 10: The Pickaxe Search Flag

FORENSIC AUDITING

Use the Git pickaxe search option to traverse the commit history and identify the exact commit hash and author that originally introduced the healthcheck API domain inside your shell script.

Hint: Look into the `-S` flag of the logging utility. Combine with short formatting or patch output flags to inspect the exact diff.

Pass Criteria: Terminal outputs the specific commit hash where string frequency changed, omitting unrelated commits.

Task 11: File Attribution (Line-by-Line Blame)

BLAME AUDITING

Run line attribution against `app_check.sh` with options that ignore whitespace noise. Verify which author and commit hash is assigned to the banner echo statement versus the exit code.

Hint: Explore `git blame`, combining it with flags that suppress whitespace alterations (`-w`).

Pass Criteria: Terminal renders tabular view showing line numbers, commit SHA, author identity, and line content.

PHASE 7: POINTER RESETS & REFLOG DISASTER RECOVERY

TASKS 12-13

Task 12: Soft Pointer Rewind Drill

RESET DRILL

Perform a soft reset to move your branch pointer backward by exactly 1 commit. Check repository status to observe how changes from the rewind commit remain staged in the index. Then, recommit the staged changes using its prior message.

Hint: Examine `git reset --soft HEAD~1` and explore the commit message reuse flag (`-c ORIG_HEAD`).

Pass Criteria: History is successfully rewind and recommitted without losing any code changes.

Task 13: Emergency Reference Log Audit

RECOVERY AUDIT

Inspect your local reference log to locate the history of all recent `HEAD` pointer movements (resets, branch switches, checkouts). Identify the commit index state before your reset drill took place.

Hint: The reference log engine tracks every state change via `HEAD@{n}` syntax. Check `git reflog`.

Pass Criteria: Reflog displays chronological index movements documenting resets, merges, and checkouts.

Task 14: Report Compilation (GIT_TASK_REPORT.md)

DOCUMENTATION

Fill out your documentation report file inside `submissions/<your_username>/GIT_TASK_REPORT.md` adhering strictly to the template structure outlined below. Include terminal outputs from your verification tasks.

Hint: Include real terminal outputs captured during your session. Stage and commit the report with a `docs: prefix`.

Pass Criteria: Report file is committed within your personal folder; no root files are altered.

Task 15: Upstream Alignment & Pull Request Submission

PR LIFECYCLE

Fetch the latest changes from `upstream`. Rebasing your feature branch on top of `upstream/main` to maintain a clean linear history. Push your branch to `origin`, navigate to GitHub, and open a Pull Request against `Sukhilkumarnatha/DevOps_Tasks:main`.

Hint: Use `git fetch upstream`, **rebase with** `git rebase upstream/main`, and **publish using** `git push -u origin <branch>`.

Pass Criteria: Pull request is opened cleanly without merge conflicts, comparing your fork's branch to Sukhil's repository.

Required Structure for `submissions/<username>/GIT_TASK_REPORT.md`

```
# Git & DevOps Practical Task Report
**Student Name:** [Your Full Name]
**GitHub Username:** [Your Username]
**Branch Name:** feature/git-practice-[your_username]

### 1. Dual Remote Verification
> Paste output of verbose remote listing showing origin and upstream.

### 2. Commit Graph & Forensics
> Paste output of single-line ASCII graph log showing recent commits.
> State the commit hash identified using the pickaxe search flag.

### 3. Conflict Resolution Summary
> Write a brief 2-sentence explanation of how you resolved the merge conflict in app_check.sh.

### 4. Reference Log Snapshot
> Paste top 4 entries from your local reflog documenting your reset drill.
```

Evaluation Rubric & PR Acceptance Criteria

Audit Criteria	Verification Standard	Evaluation
Folder Isolation	All committed assets are strictly confined to <code>submissions/<username>/</code> .	Pass / Fail
Commit Discipline	Commits follow Conventional Commits formatting (<code>feat</code> , <code>fix</code> , <code>docs</code> , <code>merge</code>).	Pass / Fail
File Permissions	<code>app_check.sh</code> preserves executable permissions bit (<code>100755</code>).	Pass / Fail
Report Accuracy	All 4 verification sections of <code>GIT_TASK_REPORT.md</code> contain genuine CLI outputs.	Pass / Fail
Clean Upstream PR	PR merges cleanly against <code>Sukhilkumarnatha/DevOps_Tasks:main</code> without conflicts.	Pass / Fail

Final Reminder: Before opening your PR, always run `git status` to ensure zero stray files or untracked artifacts exist in your workspace.